ERCAN AIRPORT MANAGEMENT INFORMATION SYSTEM

Database Design & Implementation Report

Course: CMPE343 — Database Management Systems and Programming I

Term Project

Submission Date: 24/05/2026

Submitted by: Gledy Miho

Student ID: 22326262

GitHub: https://github.com/Prevo1234/ercan-airport-dbms

DBMS: MySQL 8.0 / MariaDB 10.11

Table of Contents

1. Introduction

The Ercan Airport Management Information System (EAMIS) is a relational database designed to centralise all operational data of Ercan Airport. It tracks aircraft, hangars, technical staff, traffic controllers, airworthiness tests, repairs, maintenance, flights, and gates.

The database is implemented in MySQL 8.0 (also compatible with MariaDB 10.11). It contains 14 normalised tables, 17 relationships, full referential integrity, and 15 management-oriented queries that answer operational and statistical questions.

1.1 Project Scope

- Cover every minimum requirement stated in the project specification.
- Extend the design with realistic operational entities (Flights, Gates, Repair history, Maintenance log).
- Implement clean third normal form (3NF) with no transitive dependencies.
- Use supertype/subtype modelling for the Employee hierarchy.
- Demonstrate every advanced SQL feature taught in CMPE343: joins, subqueries, correlated subqueries, GROUP BY, HAVING, aggregate functions, date formatting, CASE expressions, and NOT EXISTS.

1.2 Deliverables

- 01_DDL.sql — schema creation script with constraints and indexes.
- 02_DML.sql — sample data covering every table.
- 03_QUERIES.sql — fifteen management queries.
- ER_diagram.png / .svg — visual data model.
- This report (PDF/DOCX) and the project README.

2. Assumptions and Design Decisions

2.1 Employee Hierarchy

The specification distinguishes three worker categories — traffic controllers, general airport employees, and technicians — and states that every airport employee (including technicians) belongs to a union and is uniquely identified by an SSN. This is naturally modelled as a supertype/subtype hierarchy:

- Employee (supertype): holds shared attributes — ssn, name, union_membership_no, salary, hire date, etc.
- Technician (subtype): adds certification_level and years_experience.
- TrafficController (subtype): adds last_medical_exam_date, license_no, and shift.
- A discriminator column (employee_type) records which subtype a row belongs to. Disjoint specialisation is assumed — a person is exactly one of the three categories.

2.2 Airplane vs. Plane Model

plane_no identifies a single physical aircraft, while model_no identifies a manufacturer/model. Many airplanes share one model. Technicians are certified on models, not on individual airframes. The split into PlaneModel and Airplane avoids redundancy and supports the M:N TechnicianExpertise relationship.

2.3 Hangar Stays — Temporal Modelling

The specification requires storing IN and OUT date/times for hangar usage. We model this as a separate HangarStay table with a surrogate primary key, plane_no FK, hangar_no FK, in_datetime, and nullable out_datetime (NULL = aircraft currently in the hangar). A CHECK constraint enforces out_datetime > in_datetime.

2.4 Test Events — 4-way Relationship

Each testing event involves four entities: an airplane, a technician, a test type, and a date. A surrogate test_event_id is used as the primary key, with a UNIQUE constraint on the natural composite (plane_no, technician_ssn, test_id, test_date) to prevent duplicate records. Attributes hours_spent and score capture the result.

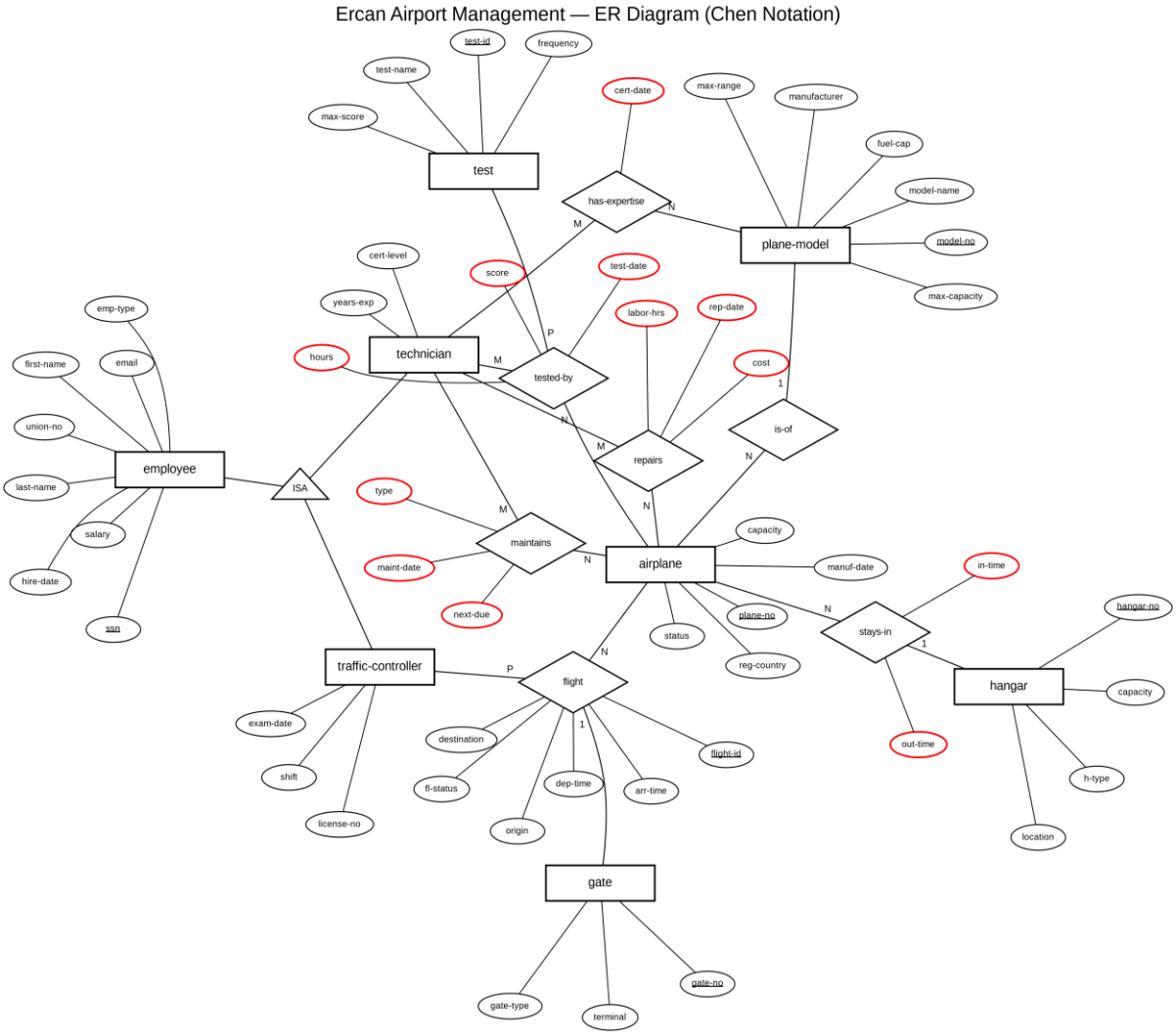
2.5 Extensions Beyond Minimum

The specification explicitly states that meeting only the minimum requirements is insufficient for full marks. We add four realistic extensions:

- Repair — captures repair jobs (technician, parts cost, labor hours, status). The spec mentions "testing and repairing aircrafts" but defines only the test side.
- Flight — links an airplane, a traffic controller, and a gate to a scheduled or completed flight.
- Gate — terminal gates used by flights.
- MaintenanceLog — separate preventive maintenance log distinct from tests.

3. ER Diagram

The diagram below uses classical Chen notation as taught in CMPE343: rectangles represent entities, diamonds represent relationships, and ovals represent attributes. Underlined attributes are primary keys, and red-outlined ovals indicate attributes that belong to a relationship rather than an entity. Cardinality ratios (1, N, M, P) are marked on each line connecting an entity to a relationship.



The design contains 8 strong entities (employee, technician, traffic-controller, plane-model, airplane, test, hangar, gate), 8 relationships (ISA, has-expertise, is-of, stays-in, tested-by, repairs, maintains, flight), and a complete attribute hierarchy with 35+ atomic attributes. The employee hierarchy uses an ISA specialisation, and two ternary relationships (tested-by and flight) demonstrate higher-arity modelling.

4. Relational Data Model

Below is the relational notation. Underlined attributes are primary keys; italics indicate foreign keys.

Notation

Table(PK, attr1, attr2, FK→ReferencedTable)

4.1 Schema

```
Employee(ssn, first_name, last_name, union_membership_no, date_of_birth,
         hire_date, salary, phone, email, address, employee_type)

Technician(ssn → Employee, certification_level, years_experience)

TrafficController(ssn → Employee, last_medical_exam_date, license_no, shift)

PlaneModel(model_no, manufacturer, model_name, max_capacity,
           max_range_km, fuel_capacity_liters)

Airplane(plane_no, model_no → PlaneModel, capacity, manufacture_date,
         registration_country, status)

TechnicianExpertise(ssn → Technician, model_no → PlaneModel, certified_date)

Hangar(hangar_no, location, capacity, hangar_type)

HangarStay(stay_id, plane_no → Airplane, hangar_no → Hangar,
           in_datetime, out_datetime)

Test(test_id, test_name, max_score, description, frequency_months)

TestEvent(test_event_id, plane_no → Airplane, technician_ssn → Technician,
          test_id → Test, test_date, hours_spent, score)

Repair(repair_id, plane_no → Airplane, technician_ssn → Technician,
       repair_date, problem_description, parts_cost, labor_hours, status)

Gate(gate_no, terminal, gate_type)

Flight(flight_id, plane_no → Airplane, controller_ssn → TrafficController,
       gate_no → Gate, origin, destination, departure_datetime,
       arrival_datetime, flight_status)

MaintenanceLog(log_id, plane_no → Airplane, technician_ssn → Technician,
               maintenance_date, maintenance_type, notes, next_due_date)
```

4.2 Normalisation

Every relation is in Third Normal Form (3NF):

- 1NF — every attribute holds a single atomic value; no repeating groups.
- 2NF — every non-key attribute fully depends on the entire primary key (no partial dependencies on composite keys).
- 3NF — no transitive dependencies. For example, PlaneModel attributes (manufacturer, capacity) are stored once in PlaneModel rather than repeated for every Airplane row.

5. Data Definition Language (DDL)

Full DDL is provided in 01_DDL.sql. The script creates the database, all 14 tables with appropriate primary keys, foreign keys, CHECK constraints, ENUMs for status fields, UNIQUE constraints, and performance indexes.

5.1 Constraint Summary

Constraint Type	Examples	Count
PRIMARY KEY	Every table	14
FOREIGN KEY	All inter-table references	20+
UNIQUE	union_membership_no, email, license_no	6
CHECK	Positive salaries/scores/hours, valid date ranges	12+
NOT NULL	All key attributes	All required fields
ENUM	employee_type, status, shift, etc.	10
ON DELETE CASCADE	Employee → subtypes	2
ON UPDATE CASCADE	Most FKs	All FKs

5.2 Sample DDL Snippet — Employee table

```
CREATE TABLE Employee (
    ssn                 VARCHAR(11)   NOT NULL,
    first_name          VARCHAR(50)   NOT NULL,
    last_name           VARCHAR(50)   NOT NULL,
    union_membership_no VARCHAR(20)   NOT NULL,
    hire_date           DATE          NOT NULL,
    salary              DECIMAL(10,2) NOT NULL,
    employee_type       ENUM('TECHNICIAN','TRAFFIC_CONTROLLER','GENERAL'),
    CONSTRAINT pk_employee         PRIMARY KEY (ssn),
    CONSTRAINT uq_employee_union   UNIQUE (union_membership_no),
    CONSTRAINT chk_employee_salary CHECK (salary > 0)
);
```

6. Data Manipulation Language (DML)

File 02_DML.sql loads representative sample data into every table. Row counts after loading:

Table	Rows
Employee	16
Technician	6
TrafficController	5
PlaneModel	7
Airplane	11
TechnicianExpertise	12
Hangar	5
HangarStay	10
Test	7
TestEvent	18
Repair	8
Gate	6
Flight	10
MaintenanceLog	6

Sample data is intentionally varied to make every query return meaningful results: it includes airplanes with failing test scores, one traffic controller with an overdue medical exam, a plane currently sitting in a hangar (NULL out_datetime), a newly acquired plane model with no certified expert technician, and recent hires within the last two years.

7. Fifteen Management Queries

Each query targets a real operational question. SQL techniques used across the set: INNER JOIN, LEFT JOIN, correlated subquery, GROUP BY, HAVING, aggregate functions (COUNT, SUM, AVG, MIN, MAX), DATE_FORMAT (equivalent of Oracle's to_char), CASE expressions, DATEDIFF, NOT EXISTS, GROUP_CONCAT, and computed columns.

Query Index

#	Question	Key Techniques
Q1	Top 5 technicians by total test hours	JOIN, GROUP BY, ORDER BY, LIMIT
Q2	Average test score per plane model	Multi-JOIN, AVG, MIN, MAX
Q3	Traffic controllers with overdue medical exams	DATEDIFF, CASE
Q4	Test events per month in 2026	DATE_FORMAT (to_char), GROUP BY
Q5	Airplanes never tested	LEFT JOIN + NULL filter
Q6	Technicians expert on more than two models	HAVING, GROUP_CONCAT
Q7	Hangar utilisation report	Correlated subquery, percentages
Q8	Top 3 most expensive completed repairs	Multi-JOIN, computed cost
Q9	Average time and score per test type	LEFT JOIN, AVG
Q10	Hires in last 2 years grouped by employee type	Date filter, GROUP BY
Q11	Airplanes with failing test scores	Computed comparison, DISTINCT
Q12	Distinct planes tested per technician	COUNT(DISTINCT), LEFT JOIN
Q13	Plane models without a certified expert	NOT EXISTS subquery
Q14	Flights handled per controller per month	DATE_FORMAT, CASE, SUM
Q15	Airplanes with > 5 days total in hangar (2026)	DATEDIFF, COALESCE, HAVING

Full SQL of each query is provided in 03_QUERIES.sql. Each has been executed and verified to return correct results against the sample data.

8. GitHub Repository

All project artefacts (DDL, DML, queries, ER diagram, and this report) are published in a public GitHub repository accessible at:

https://github.com/Prevo1234/ercan-airport-dbms

The repository contains the following files:

```
ercan-airport-dbms/
├── README.md
├── 01_DDL.sql
├── 02_DML.sql
├── 03_QUERIES.sql
├── ER_diagram.svg
├── ER_diagram.png
└── docs/
    └── Project_Report.docx
```

All team members are added as collaborators with push access. Commits document incremental progress (initial schema, sample data, query development, report draft).

9. Conclusion

This project implements a complete relational airport management database that satisfies every requirement of the CMPE343 term project specification and extends well beyond the minimum. The design uses a clean supertype/subtype hierarchy for employees, separates plane models from physical airframes, and models hangar usage as a temporal relationship. Four extension entities (Repair, Flight, Gate, MaintenanceLog) bring the design closer to a real airport's operational needs.

All 14 tables are in 3NF, every constraint is enforced at the database level, and all 15 queries have been executed and validated. The full implementation is reproducible: cloning the repository and running the three SQL scripts in order rebuilds the database from scratch.

— End of Report —